Single Translation Units: Why the Engine Assembles in One Pass

A companion guide to Building an AMIGA Game Engine

Richard Shannon

AmigaGameEngine v1.0

Contents

1. What One Translation Unit Buys **1**

2. What It Costs, and When to Link Instead **2**

> *"LINKER, n. A bureaucratic software program designed to bind independently compiled object files together, typically charging a tax of two bytes and four clock cycles on every memory transaction in exchange for the privilege."*
> *— The Bare-Metal Devil's Dictionary*

Modern practice splits a program into separate compilation units, builds each into an object file and lets a linker join them. Developers who open this repository often ask why it doesn't do the same: why `games/duck_rescue/main.asm` pulls every engine and game file in with `include` and assembles the lot in one pass, instead of building `engine.o` and `game.o` and linking them with `vlink`. This guide gives the reasons, the costs, and the cases where linking is the better choice.

1. What One Translation Unit Buys

One continuous `a5` variable block. `engine/include/engine_variables.i` starts the variable layout with `rsreset`, and the game's `variables.asm` includes it and keeps counting on the same `rs` chain (Chapter 17; Making a Game Guide §4). Because `vasm` sees both files in one pass, every field's offset is a constant at assembly time, and every access in engine or game code is a 16-bit displacement from `a5`:

```
    move.w  TickCounter(a5),d0   ; one unit, d16(An): 12 cycles, 4 bytes
    move.w  TickCounter,d0       ; separate objects, absolute long: 16 cycles, 6 bytes
```

Two separately assembled objects can't share one `rs` counter. The game either reserves a fixed-size block after the engine's (and breaks when the engine outgrows it), or the shared variables become external symbols. The usual fallback for those is absolute addressing, which costs 4 cycles and 2 bytes on every access. `vlink` can also keep base-relative access across objects with the small-data model C compilers use, but then the linker decides the layout, and you can no longer read it in the source.

Shortest branches everywhere. `vasm` knows the distance to every label in the unit, so it emits the shortest branch that reaches: `bsr.s` or `bra.s` (2 bytes) where the target is near, the word form (4 bytes) where it isn't. A call to a symbol in another object has an unknown distance at assembly time, so it is emitted in a long form and left for the linker to resolve.

Equates and flags visible everywhere. A constant, macro or `IFD` switch defined in any earlier file is visible in every later one. A game can set an option that changes how engine code assembles. Separate objects can only share what they each include from common headers.

Speed costs nothing to give up. The whole engine and game assemble and link in about 0.3 seconds on a current PC. Incremental builds were essential when a floppy-based Amiga 500 took minutes to assemble a tree this size, but they have nothing left to save here.

What a single unit does *not* change is the size of the executable's relocation table. References between sections, such as code loading the address of a Chip RAM buffer, need a `HUNK_RELOC32` entry whichever way the program

is built. The current executable has 963 of them, about 3.8 KB.

2. What It Costs, and When to Link Instead

A single unit gives up two things a linker provides:

- **Enforced encapsulation.** Every global label is visible to every file. The engine relies on discipline instead: local dot-labels (`.loop`, `.exit`) for anything internal to a routine, subsystem prefixes on global names, and `a5`/`a6` never used as scratch registers.
- **Automatic dead-code removal.** A linker can drop unreferenced sections. Here, unused code must be left out with `IFD` switches or by not including the file, which is how `games/starter/` builds without the entity system.

Separate objects and `vlink` are the right tool when:

1. **C and assembly are mixed.** The compiler and assembler each produce objects, and only the linker can join them with the C runtime.
2. **Code arrives pre-built.** A third-party library shipped as `.o` or `.a` without source can only be linked.
3. **You are writing an AmigaOS library or device.** Those are separate loadable binaries with their own conventions, built and loaded independently of any game.